

Part 1: Introduction to the Air Framework: A Fresh Breath of Python Web Development

Welcome to the first post in our comprehensive series on the Air web framework! I'm Winston Mhango, and I'll be your guide through this exciting journey into modern Python web development.

If you've been following the Python web development landscape, you might have heard whispers about a new framework that's been creating quite a buzz among developers. That framework is **Air**, a modern, FastAPI-powered web framework that's designed to breathe fresh air into Python web development.

What is Air?

Air is a cutting-edge Python web framework built on top of industry-standard libraries including FastAPI, Starlette, and Pydantic. Created by the authors of the acclaimed "Two Scoops of Django" book series, Air represents their latest innovation in web development tools.

The framework is designed with several key principles in mind:

1. **Powered by FastAPI:** Leverage all the benefits of FastAPI while adding web page generation capabilities
2. **Fast to Code:** Intuitive shortcuts and optimizations that expedite HTML development
3. **Air Tags:** A unique Python-based HTML generation system
4. **Jinja Friendly:** Seamless integration with Jinja templates
5. **HTMX Friendly:** Built-in utilities for modern HTML-based AJAX interactions

Why Air Was Created

Before diving into the technical details, it's important to understand the motivation behind Air. The creators recognized that while FastAPI excels at building APIs, creating full web applications with server-rendered HTML pages required additional work and boilerplate code. Air bridges this gap by providing:

- Simplified HTML generation through Python classes
- Easy integration of traditional web page rendering with REST APIs
- Built-in support for modern web development patterns like HTMX
- Streamlined form handling and validation with Pydantic

Core Architecture and Design Philosophy

Air's architecture is built on a foundation of four key layers:

1. **Foundation Layer:** FastAPI, Starlette, and Pydantic provide the core functionality
2. **Rendering Layer:** Air Tags and Jinja integration handle HTML generation
3. **Response Layer:** Custom response classes manage different content types
4. **Application Layer:** Air Applications and Routers provide the high-level interface

Developer Experience First

Air prioritizes making web development more intuitive and productive by reducing boilerplate code and providing powerful abstractions. Rather than reinventing the wheel, Air builds upon proven technologies, allowing developers to leverage the full power of FastAPI while adding web-specific capabilities.

Seamless Integration

Instead of forcing developers into a single paradigm, Air supports multiple approaches:

- Air Tags for Python-based HTML generation
- Jinja templates for traditional templating
- HTMX for dynamic interactions without JavaScript

Flexibility Without Compromise

Air's design allows developers to choose the right tool for each task while maintaining consistency across their codebase. Teams can mix and match approaches as needed while benefiting from Air's unified interface.

Key Features of Air

1. Air Tags - Python-Based HTML Generation

One of Air's most distinctive features is its Air Tags system. Instead of writing raw HTML or using template engines, you create HTML elements using Python classes:

```
import air

app = air.Air()

@app.get("/")
async def index():
    return air.Html(
        air.Head(
            air.Title("My Air App")
        ),
        air.Body(
            air.H1("Hello, Air!"),
            air.P("This is a paragraph created with Air Tags.")
        )
    )
```

This approach offers several advantages:

- Full IDE support with autocomplete and type checking
- Reduced context switching between HTML and Python
- Better integration with Python tooling and linting
- Type-safe HTML generation with compile-time error checking

The Air Tags system is built on the [BaseTag]

(file:///Users/macbookair/Documents/REACT%20BLOG%20APP/AIR%20PROJECTS/AIR%20TUTORIALS/helloair/air/src/air/tags/models/base.py#L44-L366) class, which provides:

- Automatic attribute handling (converting Python kwargs to HTML attributes)
- Recursive child rendering with proper text escaping
- Serialization to dictionaries and JSON for storage/transmission
- Pretty printing and browser preview capabilities

2. Seamless Jinja Integration

While Air Tags are powerful, Air also provides excellent support for Jinja templates for developers who prefer traditional templating:

```
import air

app = air.Air()
jinja = air.JinjaRenderer(directory="templates")

@app.get("/")
def index(request: air.Request):
    return jinja(request, name="home.html", title="Welcome to Air")
```

Even better, you can mix both approaches in the same project, using Jinja for overall page structure and Air Tags for dynamic fragments. The [JinjaRenderer] (file:///Users/macbookair/Documents/REACT%20BLOG%20APP/AIR%20PROJECTS/AIR%20TUTORIALS/helloair/air/src/air/templating.py#L35-L100) class automatically converts Air Tags to strings when they're included in Jinja contexts.

3. HTMX-First Approach

Air is designed with HTMX in mind, making it easy to create dynamic, modern web applications without writing JavaScript:

```
@app.page
def index():
    return air.layouts.mvpcss(
        air.H1("Counter Example"),
        air.P(
            "Count: ", air.Span("0", id="count"),
            class_="font-bold"
        ),
        air.Button(
            "Increment",
            hx_post="/increment",
            hx_target="#count",
            hx_swap="innerHTML"
```

```
)  
)
```

Air provides utilities like `[is_htmx_request]`

(<file:///Users/macbookair/Documents/REACT%20BLOG%20APP/AIR%20PROJECTS/AIR%20TUTORIALS/helloair/air/src/air/dependencies.py#L46-L47>) to detect HTMX requests and adjust responses accordingly. This enables sophisticated patterns like partial page updates and seamless navigation.

4. Pydantic-Powered Form Validation

Air leverages Pydantic for robust form validation, making it easy to create secure, validated forms:

```
from pydantic import BaseModel, Field  
import air  
  
class ContactModel(BaseModel):  
    name: str = Field(min_length=2, max_length=50)  
    email: str = Field(pattern=r"^[^@]+@[^@]+\.[^@]+$")  
  
class ContactForm(air.AirForm):  
    model = ContactModel
```

The `[AirForm]`

(<file:///Users/macbookair/Documents/REACT%20BLOG%20APP/AIR%20PROJECTS/AIR%20TUTORIALS/helloair/air/src/air/forms.py#L21-L110>) class provides:

- Automatic validation based on Pydantic field definitions
- User-friendly error messages generated from validation rules
- Flexible integration with dependency injection or direct usage in views
- Type-safe form handling with compile-time checking

Installation and Setup

Getting started with Air is straightforward. You can install it using pip:

```
pip install -U air "fastapi[standard]"
```

Or if you're using uv (recommended):

```
mkdir helloair  
cd helloair  
uv venv  
source .venv/bin/activate  
uv init
```

```
uv add air
uv add "fastapi[standard]"
```

Your First Air Application

Let's create a simple "Hello, World!" application to see Air in action:

Create a file named `main.py`:

```
import air

app = air.Air()

@app.get("/")
async def index():
    return air.layouts.mvpcss(
        air.H1("Hello, Air!"),
        air.P("Breathe it in.")
    )
```

Run your application:

```
fastapi dev
```

Visit <http://localhost:8000> to see your first Air application in action!

Advanced Features

Server-Sent Events (SSE)

Air includes built-in support for real-time communication through Server-Sent Events:

```
import random
from asyncio import sleep
import air

app = air.Air()

@app.page
def index():
    return air.layouts.mvpcss(
        air.Script(src="https://unpkg.com/htmx-ext-sse@2.2.1/sse.js"),
        air.Title("Server Sent Event Demo"),
        air.H1("Server Sent Event Demo"),
        air.P("Lottery number generator"),
        air.Section(
```

```

        hx_ext="sse",
        sse_connect="/lottery-numbers",
        hx_swap="beforeend show:bottom",
        sse_swap="message",
    ),
)

async def lottery_generator():
    while True:
        lottery_numbers = ", ".join([str(random.randint(1, 40)) for x in
range(6)])
        # Tags work seamlessly
        yield air.Aside(lottery_numbers)
        # As do strings. Non-strings are cast to strings via the str
built-in
        yield "Hello, world"
        await sleep(1)

@app.get("/lottery-numbers")
async def get():
    return air.SSEResponse(lottery_generator())

```

Layout System

Air provides pre-built layouts for rapid prototyping:

```

@app.page
async def index(is_htmx: bool = Depends(air.is_htmx_request)):
    return air.layouts.mvpcss(
        air.Title("Home"),
        air.Article(
            air.H1("Welcome to Air"),
            air.P(air.A("Click to go to Dashboard", href="/dashboard")),
            hx_boost="true"
        ),
        is_htmx=is_htmx
    )

```

The [mvpcss]

(file:///Users/macbookair/Documents/REACT%20BLOG%20APP/AIR%20PROJECTS/AIR%20TUTORIALS/helloair/air/src/air/layouts.py#L27-L93) layout function automatically includes MVP.css for basic styling and HTMX for dynamic interactions.

Air vs Flask: A Modern Comparison

For those familiar with Flask, you might wonder how Air compares. While Flask is a mature micro-framework, Air represents a more modern approach.

When comparing core foundations, Flask is built on Werkzeug and Jinja2, while Air leverages the more modern stack of FastAPI, Starlette, and Pydantic.

In terms of asynchronous support, Flask has limited capabilities in this area, whereas Air provides first-class async support out of the box.

Regarding type safety, Flask offers minimal type checking, while Air provides comprehensive type safety through its Pydantic integration.

For API development, Flask requires additional extensions to be effective, but Air has API capabilities built right into its foundation.

When it comes to HTML generation, Flask relies solely on templates, but Air offers both traditional templates and its innovative Python-based HTML generation system.

Finally, implementing modern web patterns like HTMX requires manual work in Flask, while Air has built-in support for these contemporary approaches.

Overall, Air is designed for developers who want to leverage modern Python features while maintaining the simplicity that makes Flask popular.

What's Coming in This Series

In the upcoming posts, we'll dive deeper into:

1. **Air Tags in Depth:** Mastering the Python-based HTML generation system
2. **Routing and HTTP Methods:** Building REST APIs and web pages
3. **Forms and Validation:** Creating secure, validated user input
4. **Styling with Tailwind CSS:** Making your Air applications beautiful
5. **HTMX Integration:** Building dynamic applications without JavaScript
6. **Database Integration:** Connecting Air to databases with SQLAlchemy
7. **Authentication and Security:** Implementing secure user systems
8. **React Frontend Integration:** Combining Air with modern JavaScript frameworks
9. **Mobile Development with Expo:** Bringing Air to mobile platforms

Getting Involved

Air is still an experimental framework, and the creators are actively building in public. This means you can be part of the journey:

- Check out the [source code on GitHub](#)
- Contribute to the growing ecosystem
- Join the community of early adopters

Conclusion

Air represents an exciting evolution in Python web development. By combining the power of FastAPI with intuitive tools for HTML generation and modern web patterns, it offers a fresh approach to building web applications.

In the next post, we'll dive deep into Air Tags, exploring how to create complex HTML structures using Python classes. We'll look at everything from basic elements to advanced styling and attributes.

If you're excited about this new approach to Python web development, make sure to follow this series on codetips.blog for weekly updates. You can also connect with me through [LinkedIn](#), [GitHub](#), or by email at winstonmhango23@gmail.com.

Ready to breathe some fresh air into your Python web development? See you in the next post!

This post is part of the "Mastering the Air Framework" series. Stay tuned for more deep dives into this exciting new Python web framework!

Quiz: Test Your Knowledge

1. What are the three main libraries that Air is built on top of?
 - a) Flask, Jinja2, Werkzeug
 - b) FastAPI, Starlette, Pydantic
 - c) Django, Celery, Redis
 - d) Requests, BeautifulSoup, SQLAlchemy
2. Which of the following is NOT a key feature of Air?
 - a) Air Tags - Python-Based HTML Generation
 - b) Seamless Jinja Integration
 - c) Built-in database ORM
 - d) HTMX-First Approach
3. What is the main advantage of using Air Tags over traditional HTML strings?
 - a) Faster rendering
 - b) Better compression
 - c) Full IDE support with autocomplete and type checking
 - d) Smaller file sizes
4. True or False: Air is designed with HTMX in mind, making it easy to create dynamic applications without writing JavaScript.
5. True or False: Air only supports asynchronous programming and does not support synchronous operations.
6. Briefly explain how Air's form validation works and what library it leverages for this functionality.

Answers:

1. b) FastAPI, Starlette, Pydantic
2. c) Built-in database ORM
3. c) Full IDE support with autocomplete and type checking
4. True
5. False - Air supports both synchronous and asynchronous operations

6. Air leverages Pydantic for robust form validation. Developers create Pydantic models to define form fields and validation rules, and Air provides the AirForm class to handle form rendering and validation automatically.